hdoc day 40 by trishil chauhan

Aim

To implement a menu-driven C program that performs seven student-name string operations using separate user-defined functions and pointers for output parameters.

Program (C)

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define SIZE 100
#define FULL_SIZE 300

void readText(const char prompt[], char text[], size_t capacity) {
    printf("%s", prompt);
    if (fgets(text, (int)capacity, stdin) == NULL) {
        text[0] = '\0';
        return;
    }
    size_t len = strlen(text);
    if (len > 0 && text[len - 1] == '\n')
        text[len - 1] = '\0';
    else {
        int ch;
        while ((ch = getchar()) != '\n' && ch != EOF) { }
    }
}

void combineNames(const char firstName[], const char middleName[],
                  const char lastName[], char fullName[]) {
    strcpy(fullName, firstName);
    if (middleName[0] != '\0') {
        strcat(fullName, " ");
        strcat(fullName, middleName);
    }
    strcat(fullName, " ");
    strcat(fullName, lastName);
}

void findLength(const char fullName[], size_t *length) {
    *length = strlen(fullName);
}

void copyName(const char source[], char destination[]) {
    strcpy(destination, source);
}

void compareNames(const char firstName[], const char lastName[],
                  int *comparisonResult) {
    *comparisonResult = strcmp(firstName, lastName);
}

void reverseNameOrder(const char firstName[], const char middleName[],
                      const char lastName[], char reverseName[]) {
    strcpy(reverseName, lastName);
    if (middleName[0] != '\0') {
        strcat(reverseName, " ");
        strcat(reverseName, middleName);
    }
    strcat(reverseName, " ");
    strcat(reverseName, firstName);
```

```
}

void countCharacters(const char fullName[], int *vowels,
                     int *consonants, int *digits, int *spaces) {
    *vowels = *consonants = *digits = *spaces = 0;
    for (size_t i = 0; i < strlen(fullName); i++) {
        unsigned char ch = (unsigned char)fullName[i];
        if (isalpha(ch)) {
            char lower = (char)tolower(ch);
            if (lower == 'a' || lower == 'e' || lower == 'i' ||
                lower == 'o' || lower == 'u')
                (*vowels)++;
            else
                (*consonants)++;
        } else if (isdigit(ch)) {
            (*digits)++;
        } else if (isspace(ch)) {
            (*spaces)++;
        }
    }
}

void searchWord(const char fullName[], const char word[],
                int *searchResult) {
    *searchResult = (word[0] != '\0' && strstr(fullName, word) != NULL);
}

int main(void) {
    char firstName[SIZE], middleName[SIZE], lastName[SIZE];
    char fullName[FULL_SIZE], copiedName[FULL_SIZE];
    char reverseName[FULL_SIZE], word[SIZE];
    int choice, comparison, found, vowels, consonants, digits, spaces;
    size_t length;

    readText("Enter first name: ", firstName, SIZE);
    readText("Enter middle name (blank if none): ", middleName, SIZE);
    readText("Enter last name: ", lastName, SIZE);
    combineNames(firstName, middleName, lastName, fullName);

    do {
        printf("\n--- STUDENT NAME MENU ---\n");
        printf("1. Combine and display full name\n");
        printf("2. Find length of full name\n");
        printf("3. Copy full name\n");
        printf("4. Compare first and last names\n");
        printf("5. Display reverse name order\n");
        printf("6. Count vowels, consonants, digits and spaces\n");
        printf("7. Search for a word\n");
        printf("8. Exit\nEnter choice: ");
        if (scanf("%d", &choice) != 1) {
            printf("Invalid menu input.\n");
            break;
        }
        int ch;
        while ((ch = getchar()) != '\n' && ch != EOF) { }

        switch (choice) {
            case 1:
                combineNames(firstName, middleName, lastName, fullName);
                printf("Full name: %s\n", fullName);
                break;
            case 2:
                findLength(fullName, &length);
                printf("Length: %zu\n", length);
                break;
            case 3:
                copyName(fullName, copiedName);
                printf("Copied name: %s\n", copiedName);
```

```
            break;
        case 4:
            compareNames(firstName, lastName, &comparison);
            printf("Names are %s.\n",
                   comparison == 0 ? "equal" : "different");
            break;
        case 5:
            reverseNameOrder(firstName, middleName, lastName,
                             reverseName);
            printf("Reverse order: %s\n", reverseName);
            break;
        case 6:
            countCharacters(fullName, &vowels, &consonants,
                            &digits, &spaces);
            printf("Vowels: %d, Consonants: %d, Digits: %d, "
                   "Spaces: %d\n", vowels, consonants, digits, spaces);
            break;
        case 7:
            readText("Enter word to search: ", word, SIZE);
            searchWord(fullName, word, &found);
            printf("Word %sfound.\n", found ? "" : "not ");
            break;
        case 8:
            printf("Exiting program.\n");
            break;
        default:
            printf("Invalid choice. Try again.\n");
    }
  } while (choice != 8);
  return 0;
}
```

Sample run

```
Enter first name: Trishil
Enter middle name (blank if none): Kumar
Enter last name: Chauhan

Choice 1: Full name: Trishil Kumar Chauhan
Choice 2: Length: 21
Choice 3: Copied name: Trishil Kumar Chauhan
Choice 4: Names are different.
Choice 5: Reverse order: Chauhan Kumar Trishil
Choice 6: Vowels: 7, Consonants: 12, Digits: 0, Spaces: 2
Choice 7, search Kumar: Word found.
Choice 8: Exiting program.
```

Note

The supplied prototypes for findLength, compareNames, countCharacters, and searchWord were adjusted to use pointer output parameters (e.g., size_t *length). This is necessary for call-by-reference results in C. The word search uses case-sensitive substring matching via strstr().

Result

All seven string operations are implemented using separate functions; the program exits when option 8 is selected.